

Instructor packet — airbearing (20 minutes)

Planar (x, y, yaw) GNC package for cubesats on air-bearing tables. Students describe **their** vehicle in one JSON file. **Not flight software**. SI units. Python **3.11+**. Solvers are OSQP / Clarabel via cvxpy — **no Gurobi**.

Clone: <https://github.com/AevarOfjord/airbearing-testbed-mpc> · source of truth: docs/INSTRUCTOR.md

Install (3 lines)

```
git clone https://github.com/AevarOfjord/airbearing-testbed-mpc.git
cd airbearing-testbed-mpc && python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev,viz]"
```

Then `airbearing --help` and make `test`. Windows: `.venv\Scripts\activate`.

Labs 1–4 (software; 1–2 h each)

Hardware is **not** required. make `lab1` ... make `lab4` run the staff demos. Write-ups: `labs/`. Staff notes: `labs/staff/`.

Lab	Hours	Topic	Done looks like
1	1–2 h	Editor + check	<code>airbearing check vehicles/<name>.json</code> prints both-signs YES ; JSON in <code>vehicles/*.json</code>
2	1–2 h	PD vs LQR vs MPC	Methods table from <code>airbearing report / methods.txt</code> for all three
3	1–2 h	Identify <code>F_max</code>	Identified JSON beats the guess on RMSE ; residual plot written
4	1–2 h	Binary vs PWM vs mismatch	Binary vs PWM vs <code>F_max</code> mismatch compared (make <code>lab4</code>)

Where student files go

Path	What
<code>vehicles/</code>	Their satellite JSON. Editor / new-vehicle will not save anywhere else.
<code>runs/</code>	Each run writes <code>log.csv</code> , <code>summary.json</code> , <code>methods.txt</code> , <code>plots</code> .
<code>examples/vehicles/</code>	Shipped starting guesses — copy, then calibrate. Do not edit <code>src/</code> to add a satellite.

Offline sim-vs-log demo (fresh clone, no table):

```
airbearing compare --sim examples/logs/sim.csv --real examples/logs/hardware.csv
```

Hardware (optional)

Labs 1–4 are software. A real table is extra / a later session. `--armed` is required. The runtime **refuses null mocap** (zeros commands and aborts). Gateways implement a **~100 ms deadman**. See docs/HARDWARE.md.

```
airbearing run --vehicle vehicles/mine.json --armed --port /dev/ttyUSB0 --dashboard
```

Grading — artifacts to collect

- **Lab 1:** `vehicles/*.json` plus check output showing both-signs **YES**.
- **Lab 2:** three runs/`<id>/summary.json` (or `methods.txt`) — PD, LQR, MPC on the same hop.
- **Lab 3:** identified JSON and residual plot; identified RMSE smaller than the uncalibrated guess.
- **Lab 4:** binary / PWM / mismatch numbers from make `lab4` (or equivalent `summary.json`).

Hashes in `methods.txt` (vehicle_sha256, optional git hash) make it obvious which JSON produced the run.

Safety

This kit is laboratory / teaching software, **not flight software**. The Python supervisor never *adds* thrust — it only zeros commands. Serial gateways force every actuator off if no host packet arrives within ~100 ms. `--armed` is required for hardware; null telemetry is refused rather than flown open-loop. Keep the table clear. A dashboard e-stop is not a substitute for a physical kill switch. If you bypass `--armed` or the deadman, this package will not save a runaway vehicle.